DATA CORE

Programme Omega Logistics – Rapport professionnel

Certification Data Engineer – Blocs 1 à 4 (C1-C21)

Jalon transverse M4 – Restitution BI (C16bis, C20bis)

Alexandre Laugier
20 octobre 2026

Table des matières

1 Introduction **3**

2 Bloc 1 — Cadrage (C1-C7) **4**
2.1 C1 — Analyser l'expression d'un besoin 4
2.2 C2 — Cartographier les données disponibles 4
2.3 C3 — Concevoir un cadre technique d'exploitation 4
2.4 C4 — Réaliser une veille technique et réglementaire 4
2.5 C5 — Planifier la réalisation d'un projet data 5
2.6 C6 — Superviser la réalisation d'un projet data 5
2.7 C7 — Communiquer tout au long du projet 5
2.8 Écart constaté et traité avant clôture 5

3 Bloc 2 — Collecte & Stockage (C8-C12) **6**
3.1 Séquencement retenu 6
3.2 C8 — Automatiser l'extraction de données 6
3.3 C9 — Développer des requêtes SQL d'extraction 6
3.4 C10 — Développer des règles d'agrégation de données 6
3.5 C11 — Créer une base de données 6
3.6 C12 — Partager le jeu de données 7
3.7 Point ouvert transmis au bloc suivant 7

4 Bloc 3 — Entrepôt OMEGA BI (C13-C17) **8**
4.1 Séquencement retenu 8
4.2 C13 — Modéliser un entrepôt de données 8
4.3 C14 — Créer un entrepôt de données 8
4.4 C15 — Développer des pipelines de données 8
4.5 C16 — Assurer la gestion opérationnelle d'un environnement de données 8
4.6 C17 — Mettre en œuvre une variation de dimension 9

5 Bloc 4 — Data Lake OMEGA LAKE (C18-C21) **10**
5.1 C18 — Concevoir l'architecture du *data lake* 10
5.2 C19 — Intégrer une infrastructure de données à partir de composants 10
5.3 C20 — Développer un catalogue de données 10
5.4 C21 — Garantir la gouvernance des données 11

6 Jalon M4 — Restitution BI (Grafana) **12**
6.1 Choix de l'outil 12
6.2 C16bis — Restitution de l'entrepôt OMEGA BI 12
6.3 C20bis — Monitorage du *data lake* 12
6.4 Portée budgétaire 13

7 Synthèse transverse **14**

8 Conclusion **15**

A Matrice compétences **16**

1. Introduction

2. Bloc 1 — Cadrage (C1-C7)

Le Bloc 1 pose le cadrage complet du programme DATA CORE pour Omega Logistics : analyse du besoin, cartographie des données existantes, architecture cible, veille, planification, supervision et communication. Clos le 25/08/2026 (milestone M0), les 7 compétences ont chacune un livrable produit et vérifié – détail exhaustif dans `docs/comptes_rendus/M0.md` et les documents cités ci-dessous.

2.1 C1 — Analyser l'expression d'un besoin

L'étude de faisabilité (`docs/architecture/etude_faisabilite.md`) s'appuie sur des entretiens avec les parties prenantes du programme (commanditaire opérationnel, sponsor, responsables d'entrepôt), une analyse d'opportunité et de faisabilité, une priorisation RICE des blocs de compétences, et des objectifs formulés selon la méthode SMART. Cette analyse RICE a directement arbitré le séquencement retenu pour la suite du programme (collecte avant entrepôt, entrepôt avant data lake).

2.2 C2 — Cartographier les données disponibles

La topographie des données (`docs/architecture/topographie_donnees.md`) couvre le glossaire métier et les modèles réels des trois sources identifiées – FluxPro (WMS), TransFlow (TMS) et les fichiers clients hétérogènes – en les explorant directement dans `data/raw/` et `api-mock/`, plutôt qu'en se fiant à une description théorique des sources.

2.3 C3 — Concevoir un cadre technique d'exploitation

L'étude technique d'architecture (`docs/architecture/architecture_cible.md`) établit la cible AS IS/TO BE, une matrice des flux, un registre RGPD préliminaire par brique technique, et la stratégie d'éco-conception (RGESN) et d'accessibilité (RGAA) du programme. C'est à ce stade qu'est actée la décision structurante de retenir MinIO plutôt que Microsoft Fabric pour le futur data lake : contrainte de fonctionnement local sans compte cloud, coût, reproductibilité pour un évaluateur tiers – une décision confirmée et revérifiée par la veille technique du même jour.

2.4 C4 — Réaliser une veille technique et réglementaire

Un journal de veille hebdomadaire sur trois semaines (`docs/architecture/veille_technique_reglementai`
couvre deux axes, technique (DataOps/ETL) et réglementaire (RGPD/RGAA) – chaque entrée est reliée à une décision réelle du programme, pas tenue comme un exercice isolé.

2.5 C5 — Planifier la réalisation d'un projet data

La feuille de route (`docs/architecture/feuille_de_route.md`) découpe le programme en 4 phases correspondant aux 4 blocs de compétences, avec composition d'équipe, budget prévisionnel (180 000 €) et diagrammes de Gantt/PERT – maintenus en Mermaid, c'est-à-dire en texte versionné dans le dépôt, plutôt que dans un outil graphique externe. GitHub Projects est retenu comme outil de suivi, directement intégré à la convention de branches du projet.

2.6 C6 — Superviser la réalisation d'un projet data

La supervision du programme (`docs/architecture/supervision_projet.md`) définit les rituels d'équipe (point hebdomadaire, comité de suivi mensuel) et le tableau de suivi budgétaire, actualisé à chaque jalon. Sur l'ensemble du programme (M0 à M4), le budget du périmètre initial (180 000 €) a été consommé intégralement, sans dépassement – voir le chapitre 7 pour la vue consolidée.

2.7 C7 — Communiquer tout au long du projet

Le plan de communication et les supports de lancement (`docs/architecture/communication_lancement.md`) couvrent la communication aux parties prenantes à chaque étape clé du programme, avec une attention portée à l'accessibilité des documents partagés (RGAA).

2.8 Écart constaté et traité avant clôture

Le suivi GitHub initial du Bloc 1 (issues #1 à #8) ne couvrait que 4 des 7 compétences attendues – gap identifié à la première rédaction du compte rendu de milestone, comblé par l'ouverture de 3 issues supplémentaires (C4, C6, C7) avant de considérer M0 clos. Ce réflexe de vérification — confronter le suivi du dépôt au périmètre réellement exigé, plutôt que de supposer la couverture complète — est resté une pratique constante sur l'ensemble du programme (voir notamment le réexamen du périmètre de C20 au chapitre 5).

3. Bloc 2 — Collecte & Stockage (C8-C12)

Le Bloc 2 industrialise la collecte multi-source, nettoie et consolide les données dans une base de travail, puis les expose via une API. Clos le 27/08/2026 (milestone M1), les 5 compétences ont chacune un livrable vérifié – détail exhaustif dans `docs/comptes_rendus/M1.md`.

3.1 Séquencement retenu

L'ordre C8 → C10 → C9 → C11 → C12 (`docs/architecture/sequencement_bloc2.md`), documenté et justifié avant tout développement plutôt que décidé au fil de l'eau, diffère de l'ordre naturel de lecture du cahier des charges (qui aurait suggéré de modéliser avant d'agréger). L'expérience a validé ce choix : le grain réel des doublons dans les fichiers clients n'a été découvert qu'en les traitant concrètement, pas en les modélisant par anticipation.

3.2 C8 — Automatiser l'extraction de données

Cinq connecteurs couvrent l'ensemble des sources identifiées en C2 : API REST (TransFlow), scraping du portail transporteur, requêtes SQL directes (FluxPro), fichiers clients hétérogènes, et l'historique volumineux – testés de bout en bout (31 tests).

3.3 C9 — Développer des requêtes SQL d'extraction

Cinq requêtes documentées avec leurs résultats réels (`docs/architecture/requetes_sql_extraction.md`). L'historique est chargé dans la base de staging PostgreSQL existante plutôt que dans un outil « big data » séparé : au volume réel du jeu de données (25 000 lignes), un outil dédié aurait été disproportionné — choix justifié par la sobriété numérique (RGESN) plutôt que par principe.

3.4 C10 — Développer des règles d'agrégation de données

Le nettoyage et le dédoublonnage des fichiers clients sont menés au bon grain (commande, produit) : 13 à 17 % de doublons réels mesurés sur le jeu de données, pas un chiffre théorique.

3.5 C11 — Créer une base de données

La base de travail est modélisée selon la méthode MERISE (`docs/architecture/modelisation_merise.md`), versionnée via Alembic. L'analyse des formes normales a été menée empiriquement — en interrogeant réellement les données — plutôt que théoriquement, et a révélé deux violations réelles, corrigées après vérification chiffrée : `livraisons.statut` (3NF, remplacé par une vue SQL) et `commandes_clients` (2NF, décomposée en en-tête/lignes).

La clé métier de rapprochement entre livraisons et expéditions FluxPro est `tracking_number`, portée sans contrainte de clé étrangère formelle (les deux tables appartiennent à des systèmes sources distincts) :

Listing 3.1 – Clé de rapprochement `tracking_number` (`src/datacore/storage/staging/models.py`)

```
# tracking_number : cle metier de rapprochement avec
# expeditions.tracking_number (FluxPro) -- pas de contrainte FK
# (systemes sources distincts), rapprochement fait a la lecture.
Column("tracking_number", String(20), nullable=False),
```

3.6 C12 — Partager le jeu de données

L'API Omega Data (FastAPI, choisie pour la génération automatique de la spécification OpenAPI exigée par le livrable) expose le jeu de données consolidé, avec authentification par clé et autorisation par rôle – les référents clients externes sont systématiquement restreints à leur propre client, règle reprise directement du registre RGPD établi en C11.

3.7 Point ouvert transmis au bloc suivant

Le rapprochement entre `clients.nom` (FluxPro) et le champ texte libre `historique_expeditions.client` reste non garanti par une contrainte formelle à ce stade — signalé comme point à traiter explicitement lors de la modélisation de l'entrepôt de données (chapitre 4).

4. Bloc 3 — Entrepôt OMEGA BI (C13-C17)

Le Bloc 3 construit et maintient l'entrepôt décisionnel OMEGA BI, qui répond au besoin d'analyse de la performance logistique exprimé par la direction. Clos le 21/09/2026 (milestone M2), les 5 compétences ont chacune un livrable vérifié – détail exhaustif dans `docs/comptes_rendus/M2.md`.

4.1 Séquencement retenu

L'ordre C13 → C14 → C15 → C17 → C16 (`docs/architecture/sequencement_bloc3.md`) inverse volontairement C16/C17 par rapport au Gantt initial : la procédure de purge RGPD de C16 dépend du schéma historisé que C17 doit d'abord créer.

4.2 C13 — Modéliser un entrepôt de données

Approche bottom-up par datamarts (Kimball) : 2 datamarts, 3 faits, 6 dimensions conformées, matrice de bus (`docs/architecture/modelisation_omega_bi.md`). Le seul schéma en flocon retenu (`Dim_Produit` → `Dim_Categorie`) est justifié par une contrainte réelle des sources, pas par principe de modélisation.

4.3 C14 — Créer un entrepôt de données

L'entrepôt est créé dans une base PostgreSQL distincte de la base de staging, mais sur la même instance (sobriété RGESN) – 4 schémas physiques (`dimensions`, `exploitation`, `commercial`, `gouvernance`), versionnés via un second environnement Alembic dédié. Le cycle `upgrade`/`downgrade` est vérifié contre une base réelle, pas seulement contre une base vide (un bug latent d'Alembic, `include_schemas=True` manquant, n'a d'ailleurs été révélé qu'à la seconde migration).

4.4 C15 — Développer des pipelines de données

Le pipeline ETL staging → entrepôt applique des contrôles qualité réels (mise en quarantaine des lignes non rapprochées de l'historique, pas un rejet silencieux), et sa cohérence est vérifiée en comparant les indicateurs calculés par l'ETL à ceux calculés indépendamment par l'API C12 – deux chemins de calcul distincts produisant le même résultat.

4.5 C16 — Assurer la gestion opérationnelle d'un environnement de données

La gestion opérationnelle (`docs/architecture/gestion_operationnelle_omega_bi.md`) couvre des sauvegardes planifiées (partielle/complète, justifiées par la reconstructibilité de 7 des 8 tables

depuis le staging), une journalisation réelle des opérations (succès et échec, sur une connexion dédiée pour survivre à un `ROLLBACK`), et un registre RGPD appuyé sur un audit automatisé répétable ne trouvant aucune donnée personnelle dans l'entrepôt.

Ce périmètre a été étendu après la clôture initiale du bloc : la restitution par vues SQL et notebook s'est révélée insuffisante comme preuve de restitution, et renforcée dans un outil de BI dédié – traité comme son propre jalon transverse (chapitre 6).

4.6 C17 — Mettre en œuvre une variation de dimension

`Dim_Client` est historisée en SCD2 (type 2 de Kimball) : un changement de `nom` ou `secteur` ferme la version courante et en ouvre une nouvelle, plutôt que d'écraser la valeur précédente.

Listing 4.1 – Détection du changement et historisation SCD2 (`src/datacore/storage/warehouse/load_warehouse.py`)

```
if current is not None and current[1] == r["nom"] and current[2] == r["
    secteur"]:
    mapping[r["id"]] = current[0]
    continue

if current is not None:
    cur.execute(
        """
        UPDATE dimensions.dim_client
        SET valid_to = %(today)s, is_current = false
        WHERE client_key = %(client_key)s
        """,
        {"today": today, "client_key": current[0]},
    )
# ... puis INSERT d'une nouvelle ligne avec une nouvelle client_key.
```

Le mécanisme est vérifié par un changement réel simulé : l'ancienne version reste close et interrogeable, la nouvelle est créée, et les faits déjà chargés continuent de référencer la version qui était courante au moment de leur chargement. Écart assumé avec la description initiale de l'issue (qui demandait l'historisation d'attributs absents du jeu de données pédagogique, adresse et contrat) : `nom` et `secteur` ont été retenus à la place, après décision prise avant implémentation plutôt que devinée.

5. Bloc 4 — Data Lake OMEGA LAKE (C18-C21)

Le Bloc 4 conçoit et opère le data lake OMEGA LAKE pour absorber la donnée massive IoT de la flotte et des entrepôts. Clos le 23/09/2026 (milestone M3), les 4 compétences ont chacune un livrable vérifié – détail exhaustif dans `docs/comptes_rendus/M3.md`.

5.1 C18 — Concevoir l'architecture du *data lake*

La différence entre base de staging, entrepôt et data lake n'est pas le volume ni le degré de nettoyage des données, mais le moment où le schéma est décidé : *schema-on-write* pour les deux premiers, *schema-on-read* pour le lake (`docs/architecture/architecture_omega_lake.md`). Les flux IoT transitent en flux séparé (jamais par la base de staging relationnelle), mais la zone `curated/` est conçue pour être jointe à la volée aux dimensions conformées de l'entrepôt via des clés naturelles réelles, vérifiées dans les données plutôt que supposées.

5.2 C19 — Intégrer une infrastructure de données à partir de composants

MinIO est déployé comme stockage objet compatible S3 (`docs/architecture/integration_infrastructure`). Le mécanisme de jointure — DuckDB, choisi après vérification que le projet n'utilise `pandas` nulle part — est tranché explicitement avant tout code de pipeline, et vérifié contre MinIO/PostgreSQL réels, pas supposé équivalent à un test local :

Listing 5.1 – Configuration DuckDB/`httpfs` pour MinIO (`src/datacore/storage/lake/transform.py`)

```
SET s3_endpoint='{OMEGA_LAKE_S3_ENDPOINT}';
SET s3_access_key_id='{MINIO_ROOT_USER}';
SET s3_secret_access_key='{MINIO_ROOT_PASSWORD}';
SET s3_use_ssl=false;
SET s3_url_style='path';
```

L'ingestion de `raw/` passe par `boto3` (copie fidèle, octet pour octet, vérifiée par somme de contrôle SHA-256) et non par DuckDB, qui réinterpréterait le contenu. Le consommateur du flux SSE, fenêtré par intervalle de 5 minutes plutôt que par jour, a été vérifié par une coupure brutale réelle (`kill -9` en cours de réception puis redémarrage) : aucune collision ni donnée corrompue.

5.3 C20 — Développer un catalogue de données

Le périmètre de cette compétence a été explicitement élargi avant tout code : le texte initial de l'issue ne parlait que de cataloguer « après C19 », mais à ce stade les zones `staging/`/`curated/` restaient vides – construire un contenu réel y a été ajouté. Le catalogue lui-même est généré

par introspection réelle (`boto3` + `DESCRIBE` DuckDB), pas maintenu à la main, même principe que l'audit RGPD automatisé de C16 : un catalogue généré ne peut pas devenir silencieusement obsolète.

5.4 C21 — Garantir la gouvernance des données

Le registre RGPD du lake (`docs/architecture/registre_rgpd_lake.md`) identifie un risque réel de ré-identification, trouvé en concevant et non en auditant après coup : `vehicule_id` (géolocalisation) se recoupe, dans le temps, avec `tournees.chauffeur` de la base de staging. Décision actée dès C18 : aucune jointure automatisée entre les deux.

Une première analyse de pseudonymisation (« pas de protection réelle possible, vu le faible nombre de véhicules ») s'est révélée incomplète après relecture externe : vraie pour les Data Engineers (accès complet des deux côtés), fausse pour le rôle `lake_reader` (accès au lake seul). Un pseudonyme HMAC à clé secrète — un hash simple sans clé aurait été recalculable par force brute sur les 15 véhicules du jeu de données — a été implémenté comme export dédié (`curated_bi/`), sans jamais modifier `curated/` ni distribuer la clé au rôle de lecture :

Listing 5.2 – Garde-fou sur la clé de pseudonymisation (`src/datacore/storage/lake/curated_bi.py`)

```
def verifier_cle_configuree(cle: str | None) -> str:
    if not cle:
        raise RuntimeError("LAKE_PSEUDONYM_KEY n'est pas definie. ...")
    if cle in (VALEUR_EXEMPLE_ENV, ANCIENNE_VALEUR_REPLI_SUPPRIMEE):
        raise RuntimeError("LAKE_PSEUDONYM_KEY vaut une valeur publique
            connue. ...")
    if len(cle) < LONGUEUR_MINIMALE:
        raise RuntimeError(f"LAKE_PSEUDONYM_KEY est trop courte ({len(
            cle)} caracteres). ...")
    return cle
```

Ce garde-fou a lui-même été renforcé après qu'une faille a été trouvée en seconde relecture externe — un repli codé en dur pour cette clé dans `config.py`, visible dans le dépôt public, aurait rendu la pseudonymisation silencieusement inopérante — puis à nouveau après une question sur sa couverture au-delà du seul cas de la valeur d'exemple. Les droits d'accès par groupe ont également été corrigés en cours de route : la politique `lake_reader` portait initialement sur le bucket entier, permettant de contourner la pseudonymisation en lisant `curated/` directement – restreinte au seul préfixe `curated_bi/`, revérifiée bout en bout avec les vraies clés `lake_reader` (lecture de `curated_bi/` réussit et renvoie des données pseudonymisées ; la même requête contre `curated/` échoue en HTTP 403). La procédure de purge RGPD est vérifiée par un test réel : une partition synthétique délibérément datée, injectée dans le lake réel, purgée avec succès et le résultat confirmé indépendamment.

6. Jalon M4 — Restitution BI (Grafana)

M4 n'est pas un bloc de compétences RNCP mais un jalon transverse, ajouté en cours de programme (24/09/2026) après un retour de formateur : les livrables produits pour C16 (vues SQL + notebook) et C20 (catalogue par introspection) suffisaient aux critères déjà évalués, mais pas comme démonstration de restitution/monitoring dans un outil de BI dédié. Clos le 19/10/2026, les 2 issues du jalon sont vérifiées en conditions réelles – détail exhaustif dans `docs/comptes_rendus/M4.md`.

6.1 Choix de l'outil

Grafana est retenu, auto-hébergé, plutôt que Power BI (nécessite le Power BI Service, cloud) ou Databricks SQL/Lakeview (nécessite un *workspace* Databricks) – cohérent avec l'infrastructure 100 % locale du reste du programme, décidé avant tout codage. Le notebook SLA existant n'est ni retiré ni réécrit : il reste la preuve exploratoire telle que livrée à la clôture du Bloc 3, Grafana devenant l'outil de restitution *de production* en complément.

6.2 C16bis — Restitution de l'entrepôt OMEGA BI

Les vues SQL déjà exposées pour C16 sont restituées dans un dashboard Grafana dédié, connecté en lecture seule via le rôle `bi_reader` déjà scopé en C14. Trois bugs réels ont été trouvés uniquement en testant depuis un vrai navigateur (champ requis manquant au provisioning de la source de données, mode d'édition de requête non forcé sur les panels, agrégation silencieuse masquant deux clients sur trois sur un panel) – tous invisibles depuis les vérifications par appel direct à l'API de Grafana, qui passaient toutes. Généralisé en leçon méthodologique : une vérification par API seule ne prouve jamais qu'un rendu côté client fonctionne pour l'utilisateur final.

6.3 C20bis — Monitorage du *data lake*

Une première conception du monitorage du lake, limitée au suivi de la fraîcheur des données, s'est révélée incomplète après lecture du référentiel de compétences complet (copié dans le dépôt, `docs/reference/`) : celui-ci exige explicitement le suivi des « conditions matérielles et applicatives », et qu'une alerte soit « générée lors d'une rupture de service ». Corrigé par l'ajout d'un relevé de santé de l'infrastructure (MinIO accessible, capacité utilisée) et d'une règle d'alerte Grafana réelle.

La chaîne d'alerte relie une règle Grafana, un service interne authentifié, et un relais d'e-mail réel via la bibliothèque `smtplib` vers un capteur SMTP local (MailHog) – jamais un vrai fournisseur externe, pour rester démonstrable sans identifiant réel à gérer ni envoi parasite à chaque exécution automatisée des tests. Testée de bout en bout, deux fois : une coupure réelle de MinIO déclenche un relevé en échec, l'alerte Grafana passe à l'état actif en une trentaine de secondes, et un e-mail réel est reçu dans MailHog ; la remise en service résout l'alerte. Une

relecture de code a ensuite mis au jour deux trous de robustesse (un corps de requête mal formé faisait échouer le service sans réponse exploitable, un échec d'envoi d'e-mail n'était pas distingué d'une autre erreur) et l'absence d'authentification sur ce service, exposé sur l'hôte – les trois corrigés et revérifiés avant la clôture du jalon.

6.4 Portée budgétaire

Comme les jalons transverses de rédaction du rapport et de préparation de la soutenance, M4 n'a jamais eu de ligne dédiée dans le budget prévisionnel initial (180 000 €, voir chapitre 2) – staffing interne uniquement, les outils retenus (Grafana, MailHog) étant open source et auto-hébergés. Aucun dépassement constaté.

7. Synthèse transverse

8. Conclusion

A. Matrice compétences